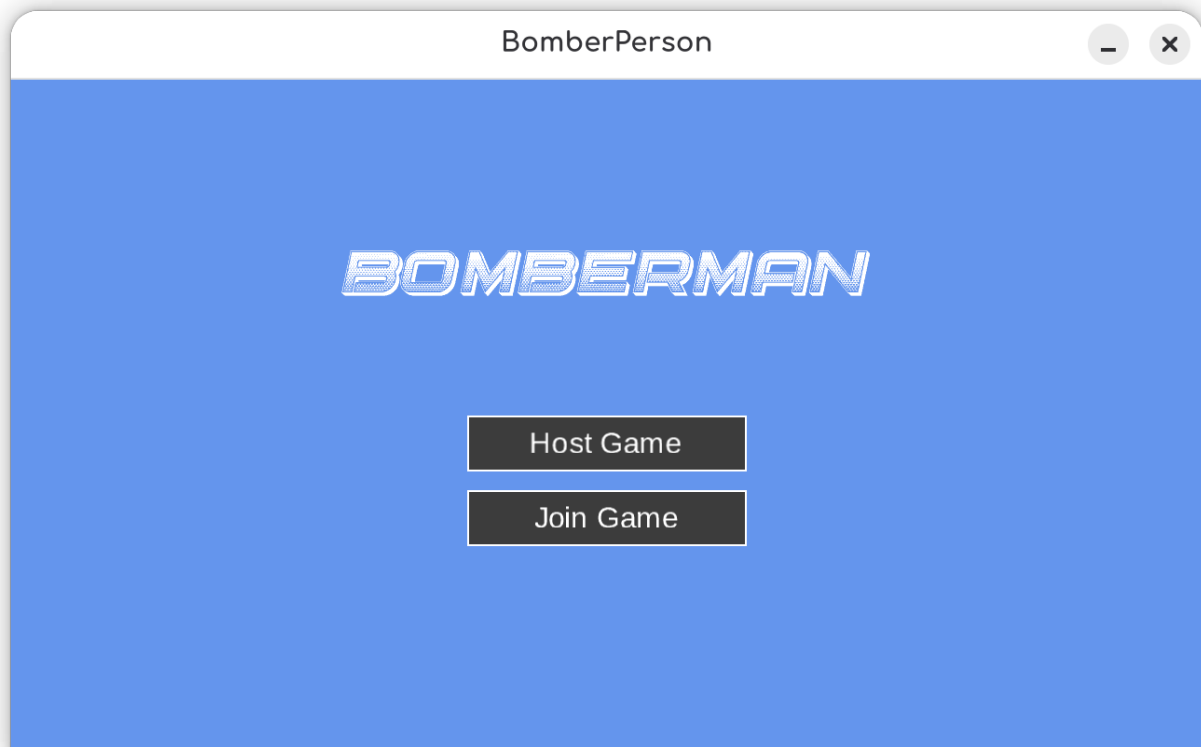


Bomber Person

Rapport PLM 2026

Raphaël Perret et Florian Duruz



Rapport

Retour d'expérience

Sur le paradigme dataflow

La prise en main du paradigme s'est faite sans difficulté particulière, et ce malgré le nombre d'objets nouveaux qu'il a fallu découvrir : *BufferBlock*, *TransformBlock*, *TransformManyBlock*, *BroadcastBlock*, *ActionBlock*, sans oublier les options de configuration (*DataflowLinkOptions*, *ExecutionDataflowBlockOptions*) et le mécanisme de propagation de complétion. Cette diversité, qui pourrait paraître intimidante au premier abord, est en réalité un atout : chaque bloc a une responsabilité unique et clairement définie, ce qui permet de composer un pipeline en choisissant les briques adaptées à chaque étape.

J'ai trouvé l'implémentation du paradigme par la bibliothèque TPL Dataflow particulièrement propre. L'API est cohérente, tous les blocs partagent les mêmes interfaces (*ISourceBlock*, *ITargetBlock*, *IPropagatorBlock*), ce qui rend la composition triviale et permet aussi d'écrire ses propres blocs lorsque les blocs standards ne suffisent pas (cela a été notre cas avec *RealisationDateBlock*, qui s'insère dans le pipeline exactement comme un bloc natif). Le mécanisme de **backpressure**, la gestion des erreurs via *Fault*, la propagation de la complétion : tout est pensé et fourni d'office, on n'a pas eu à réinventer ces aspects-là.

Un autre point fort tient à la **séparation des responsabilités** que le paradigme impose presque mécaniquement. La structure même d'un pipeline force à découper le traitement en étapes distinctes communiquant uniquement par messages, sans état partagé. Dans notre cas, la simulation, la diffusion aux clients et l'ordonnancement des événements futurs sont trois blocs distincts qui ignorent tous les uns des autres ; les remplacer ou les tester en isolation devient presque trivial. C'est un type de découplage qu'il faut habituellement discipliner soi-même dans une architecture conventionnelle, et qu'on obtient ici par construction.

En revanche, sur la **plus-value réelle** du paradigme, je dois admettre qu'il m'est difficile de me prononcer fermement. Le projet, tel qu'il a évolué, n'a pas mis en avant le dataflow autant que je l'avais imaginé au départ : notre pipeline est relativement linéaire, avec un seul vrai point de fan-out (le broadcast vers les clients) et une seule boucle de rétroaction (les événements programmés). On n'exploite ni le parallélisme massif que permettent les `MaxDegreeOfParallelism > 1`, ni des topologies plus riches (fan-in, agrégation, pipelines parallèles). Beaucoup des bénéfices que le paradigme peut offrir : débit élevé sur traitement indépendant, composition de transformations sur des flux continus ; ne s'expriment pas vraiment dans un netcode de jeu où l'autorité est centralisée et où le débit reste modeste.

Si je me pose la question honnêtement **"est-ce que je choisirais à nouveau ce paradigme plutôt qu'un autre pour un projet comparable ?"**, ma réponse est non. Non pas qu'il soit inadapté : il fonctionne très bien et le code final est lisible. Mais il n'apporte pas assez de plus-value, pour ce type de problème, pour justifier le coût d'apprentissage d'un paradigme complet là où une architecture conventionnelle (boucle de jeu serveur classique, sockets gérés avec des *Task* et quelques *Channel<T>* ou *ConcurrentQueue*) aurait abouti à un résultat fonctionnellement équivalent, avec moins de concepts nouveaux à intégrer. Le dataflow brillerait certainement

davantage sur un problème dont la structure est intrinsèquement un graphe de transformations : pipeline ETL, traitement d'images, agrégation de flux temps réel; plutôt que sur un netcode où le besoin principal reste « *recevoir un input, muter un état, renvoyer un snapshot* ».

Sur le langage C#

Mon retour sur C# est nécessairement biaisé : je l'utilise depuis plus de cinq ans, je m'y considère à l'aise, et je n'ai donc pas vécu ce projet comme une découverte du langage. À ce titre, je dois reconnaître d'emblée que je n'ai pas le recul d'un nouvel arrivant pour pointer ses aspérités, ce qui me paraît naturel relèverait probablement de l'apprentissage pour quelqu'un qui en ferait l'expérience pour la première fois.

Cela posé, je n'ai à peu près que du bien à en dire. C# est souvent comparé à Java, et il est vrai que les deux langages partagent une syntaxe et un modèle objet très proche. Mais C# bénéficie d'un panel de ****sucre syntaxique**** qui le rend, à mon sens, nettement plus agréable à écrire et à lire au quotidien. L'exemple le plus parlant, et c'est probablement ce que j'apprécie le plus dans le langage reste les ****propriétés automatiques**** : une déclaration comme ``public int SlotId { get; private set; }`` remplace les deux méthodes ``getSlotId()`` / ``setSlotId()`` que Java imposerait, sans rien sacrifier de leur sémantique (encapsulation, visibilité différenciée, possibilité de remplacer ultérieurement par une implémentation explicite). Sur une codebase de taille modeste comme la nôtre, l'économie cumulée en lignes de code et en bruit visuel est déjà appréciable ; sur un projet plus large, le gain est substantiel.

Au-delà des propriétés, l'ensemble des facilités syntaxiques offertes par les versions modernes du langage (expressions ``switch``, ***pattern matching***, ***records***, ***target-typed new***, ``using`` declarations, ***primary constructors***, etc.) concourent à un même résultat : exprimer l'intention plutôt que la mécanique. C'est pour moi l'argument principal en faveur de C#, et la raison pour laquelle, à choix égal, je le préférerai presque systématiquement à un langage équivalent fonctionnellement mais plus verbeux.

Concernant ce projet précisément, le langage n'a posé aucun obstacle : la bibliothèque TPL Dataflow s'intègre naturellement avec ``async`/`await`` et le système de types fort, le compilateur est suffisamment strict pour rattraper la plupart des erreurs avant l'exécution, et les outils (IDE, débogueur) sont parmi les meilleurs que j'aie utilisés. Le choix de C# pour explorer un paradigme dataflow s'est avéré pertinent, ne serait-ce que parce que la bibliothèque de référence pour ce paradigme est précisément intégrée à la BCL.

Architecture du projet

Vue d'ensemble

Le jeu suit un modèle client-serveur autoritaire : le serveur est l'unique source de vérité de l'état de la partie, et tous les clients ne font qu'envoyer leurs intentions et afficher l'état que le serveur leur renvoie. Cette séparation est volontaire : elle évite toute logique de jeu côté client (et donc toute désynchronisation à résoudre), et elle correspond naturellement au paradigme dataflow : un flux unidirectionnel de messages traversant une chaîne de blocs.



Le transport utilisé est TCP, via les classes `TcpListener` et `TcpClient` de `System.Net.Sockets`. TCP a été retenu pour sa fiabilité et sa simplicité. Il permet sans effort supplémentaire de ne jamais perdre une intention d'un joueur.

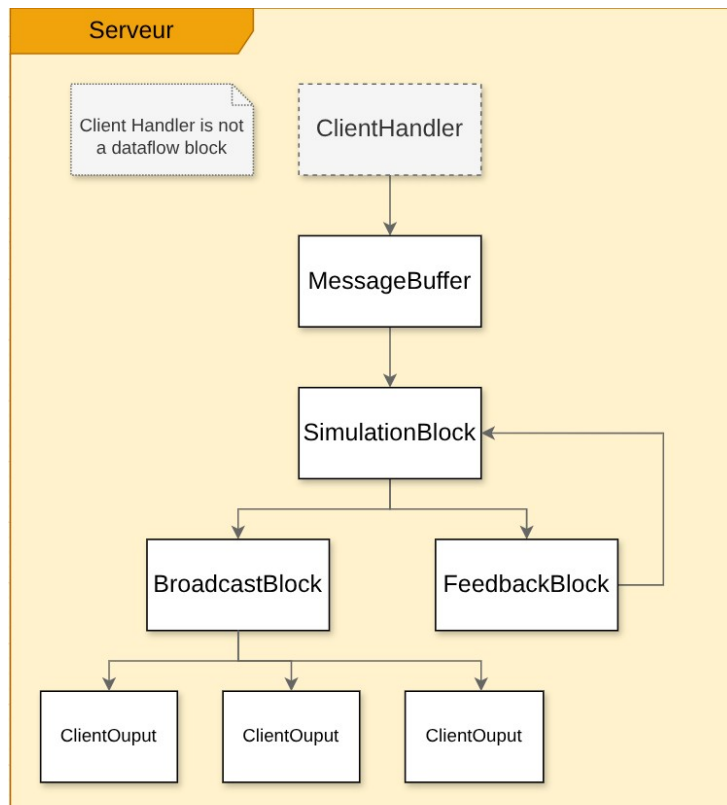
Lorsqu'un joueur héberge une partie, le même processus exécute à la fois un serveur et un client automatiquement connecté. Le code du client est donc strictement identique pour l'hôte et pour les joueurs distants, ce qui simplifie l'architecture : il n'existe qu'un seul chemin d'exécution côté client. Le serveur traite tous les clients de manière égale et peut être facilement utilisé comme serveur standalone sur lequel tous les clients se connectent.

Le pipeline dataflow

Nous n'avons finalement pu intégrer dataflow que sur le serveur. Lorsque le serveur est lancé, il construit le pipeline pour traiter les messages et y répondre. Le pipeline se constitue de :

- **messageBuffer** (*BufferBlock<IMessage>*) : point d'entrée unique du pipeline. Tous les messages reçus des clients y sont postés, et ce quel que soit le client émetteur. Le bloc sérialise naturellement les producteurs concurrents en une file ordonnée.
- **simulationBlock** (*TransformManyBlock<IMessage, IMessage>*) : exécute la logique métier via `Simulation.ProcessMessage`. Traiter un message va toujours renvoyer un nouvel état à tous les clients connectés, mais peut en renvoyer plusieurs.
- **broadcastBlock** (*BroadcastBlock<IMessage>*) : diffuse chaque message sortant à tous les abonnés. Chaque client connecté s'y abonne lors de son arrivée, ce qui permet de réaliser une diffusion **one-to-many** sans que la simulation n'ait à connaître les clients.
- **feedbackBlock** (*RealisationDateBlock<IMessage>*) : bloc personnalisé qui retient un message jusqu'à une date donnée, puis le réinjecte dans `simulationBlock`. Il sert à programmer des événements futurs (fin de partie, explosion d'une bombe, collision prévisible) sans timers externes : la simulation décrit **quand** un événement doit survenir, et le pipeline le réinjecte au bon moment.

Ce pipeline est conservé pour toute la durée de vie du serveur. Des blocs d'actions sont ajoutés au broadcast block lorsque des clients se connectent.



Réception et envoi des messages

Notre espoir dans l'utilisation de dataflow était de pouvoir encapsuler le client tcp dans un bloc source, qui produirait spontanément des messages lorsque le socket les reçoit. En réalité, même un bloc source ne peut pas générer des données : elles doivent à un moment ou un autre être injectées dans le pipeline par du code séquentiel. Dans ce cas, on doit donc manuellement créer une tâche asynchrone pour chaque client qui se connecte. Nous le faisons avec la classe *ClientHandler*, qui dans sa méthode *handle* crée le bloc de sortie et le lie au broadcast, place un message annonçant l'arrivée d'un nouveau joueur puis ne fait plus que lire les messages arrivant sur le réseau.

Dataflow nous permet de simplement ajouter de nouveaux messages à traiter de manière asynchrone, d'effectuer des transformations dessus puis d'envoyer les réponses.

Messages

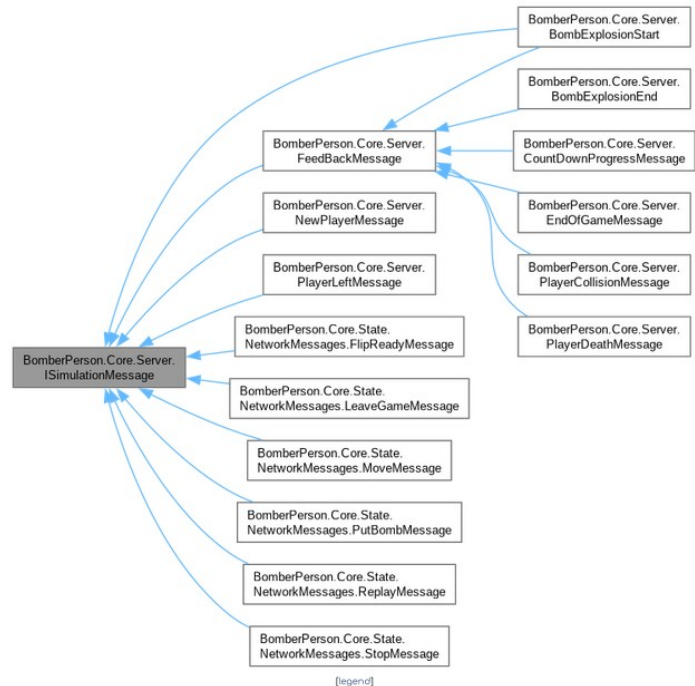
Une interface *Imessage* permet de passer des messages quelconques dans le pipeline. On la spécialise en messages pouvant être envoyés sur le réseau (*NetworkMessage*), message impactant la simulation (*SimulationMessage*), et messages que la simulation s'adresse à elle-même.

Les messages réseau implémentent une méthode *serialize*, et peuvent être produits par une fabrique de messages réseau, pour passer vers ou depuis une représentation binaire du message.

Les messages de la simulation ont une date de réalisation, qui correspond au moment où un évènement se produit, comme une collision ou une explosion. Ils contiennent aussi les données relatives à cet évènement.

Les clients communiquent au serveur avec toute une panoplie de messages différents, mais le serveur renvoie presque exclusivement un message réseau contenant le nouvel état du jeu. Ces derniers prennent entre 600 et 1000 bytes lors d'une partie en cours.

Pour aller plus loin, l'envoi de l'état devrait être grandement optimisé avec notamment la possibilité de ne recevoir que les zones de la grille qui ont été modifiées par exemple. Même si un joueur ne fait que s'arrêter et que le terrain n'est pas modifié, la grille complète est envoyée, soit 400 bytes pour la grille 20x20 par défaut.



Activité sur le réseau

Les clients envoient une multitude de messages ne contenant que peu d'information. Mais pour chaque message d'un client traité, le serveur renvoie l'état complet. Cela fonctionne surprenamment bien pour un jeu dont le monde peut être représenté dans moins d'un kilobyte. On notera aussi que si les clients restent inactifs, aucun packet n'est envoyé. La bande passante n'est dans notre application pas un problème.

En revanche la latence est une autre histoire. En démarrant plusieurs clients sur la même machine, elle ne se remarque pas et le jeu est parfaitement jouable. Sur une connection câblée assez directe elle reste acceptable, mais dès qu'il est question de wifi ou cellulaire, la latence rend le jeu difficile pour le joueur distant. Rappelons que lorsque le joueur presse sur une touche pour avancer, l'affichage ne change pas. Un message *MoveMessage* est envoyé et seulement lorsque le serveur répond, le client mets l'état à jour avec le joueur en mouvement.

Remarques sur la simulation

La simulation est séparée en plusieurs fonctions. *Interpolate* ne fait qu'une interpolation linéaire des positions des joueurs à partir de la date de création de l'état. Elle est utilisée par le serveur pour calculer l'état actuel et par le client pour interpoler. *NextEvent* calcule le prochain évènement qui va se produire si aucun message ne vient des clients. Certains sont triviaux à déterminer, comme les explosions de bombes, la fin de la partie par manque de temps, ... mais les collisions sont plus compliquées et requièrent d'avancer peu à peu dans le temps pour ne pas rater de collision. On détermine donc d'abord l'évènement trivial le plus proche et on calcule les collisions jusqu'à cet évènement. (En jeu, on calcule rarement les collisions plus d'une seconde dans le futur). De plus, il est inutile de continuer si tous les joueurs sont arrêtés. Par manque de temps, le temps en chaque itération a été réduit pour obtenir une meilleure précision de manière triviale,

mais la bonne solution serait de faire des pas plus grands, puis rechercher la date de la collision à une précision arbitraire par recherche dichotomique.

Synthèse

L'architecture repose sur un principe simple : ****les messages entrent par un point unique, traversent un pipeline déterministe, et ressortent vers tous les clients via un broadcast****. Les sockets TCP ne sont jamais manipulées directement par la logique métier ; elles sont encapsulées par des blocs dataflow (un `ActionBlock` par sens et par socket), ce qui garantit qu'à tout instant un seul thread écrit dans chaque flux. La synchronisation, traditionnellement source de bugs dans une application multi-clients, est ici une conséquence naturelle des contraintes de parallélisme posées sur chaque bloc, et non un sujet à traiter explicitement avec des verrous.

